

Software Requirements Specification (SRS)

UrbanPulse

UrbanPulse – Smart Urban Waste Management System

1. Introduction

1.1 Purpose

UrbanPulse is a web-based waste management platform designed to improve communication between citizens, sanitation workers, and municipal authorities. The system enables users to report waste-related issues, manage daily waste collection operations, track complaint resolution, and monitor waste collection activities through an interactive map.

The purpose of this project is to provide a centralized platform that enhances transparency, accountability, and efficiency in urban waste management.

1.2 Scope

UrbanPulse addresses common urban sanitation problems such as:

- Illegal waste dumping
- Overflowing public bins
- Missed door-to-door waste collection
- Damaged public waste bins
- Inefficient complaint tracking

The platform enables citizens to report issues, allows administrators to assign workers, and provides workers with their daily collection routes and assigned tasks.

1.3 Intended Audience

- Municipal Authorities
- Waste Management Agencies
- Sanitation Workers
- Citizens
- Developers
- Project Evaluators

1.4 Definitions

Term	Description
Concern	A waste-related issue reported by a citizen
Assignment	Task assigned to a sanitation worker
Collection Route	Daily route assigned to a worker
Collection Point	Individual pickup location on a route
Admin	Municipal authority managing the system
Citizen	User reporting issues
Worker	Waste collection personnel

2. Overall Description

2.1 Product Perspective

UrbanPulse is a standalone web application consisting of:

- React Frontend
- FastAPI Backend
- PostgreSQL Database
- Cloudinary Image Storage
- Interactive Map Integration

The application acts as a communication bridge between citizens and municipal authorities.

2.2 Product Features

Major features include:

- User Authentication
- Role-based Access Control
- Concern Reporting
- Interactive Map
- Image Upload
- Worker Assignment
- Daily Collection Routes
- Collection Point Management
- Notifications

- Suggestion System
- Dashboard & Analytics
- AI Chatbot (Future/Optional)

2.3 User Classes

Guest

- View homepage
- Report concern

Citizen

- Register/Login
- Report concerns
- Upload images
- Report missed collection
- Add map pin
- View concern status
- Submit suggestions
- Receive notifications

Worker

- Login
- View assigned routes
- Mark collection completed
- Report collection issues
- Resolve assigned concerns
- Upload completion proof

Administrator

- Assign workers
- Manage collection routes
- Manage collection points
- Review suggestions

- View analytics
- Monitor complaints

3. Functional Requirements

Authentication

- User registration
- Secure login
- JWT authentication
- Role-based authorization

Concern Management

- Create concern
- Upload images
- View concern status
- Edit/Delete concern before assignment
- Support existing concerns

Worker Management

- View assigned work
- Accept assignments
- Complete assignments
- Upload proof images

Collection Route Management

- Create daily routes
- Assign workers
- View collection points
- Update collection status

Collection Point Management

- Mark collection completed
- Report issue

- Specify issue reason
- Track completion time

Notifications

- Concern assigned
- Concern resolved
- Route assigned
- Suggestion accepted
- Missed collection updates

Suggestions

- Submit suggestions
- Admin review
- Accept or reject suggestions

Dashboard

Admin Dashboard

- Total users
- Active workers
- Pending concerns
- Resolved concerns
- Route statistics

Worker Dashboard

- collection route
- Assigned concerns
- Completed collections

Citizen Dashboard

- Submitted concerns
- Notifications
- Concern history

4. Non-Functional Requirements

Performance

- API response time under 2 seconds
- Efficient database queries
- Concurrent user support

Security

- JWT Authentication
- Password hashing
- Role-based authorization
- Input validation
- SQL Injection protection

Reliability

- Daily database backup
- Error logging
- Exception handling

Scalability

System should support:

- Multiple wards
- Multiple workers
- Thousands of concerns
- Future mobile application

Availability

Target availability:

99% uptime during deployment.

Maintainability

- Modular architecture
- Layered backend
- API documentation
- Clean code practices

5. Technologies Used

Frontend

- React
- Tailwind CSS
- Axios
- React Router
- React Leaflet

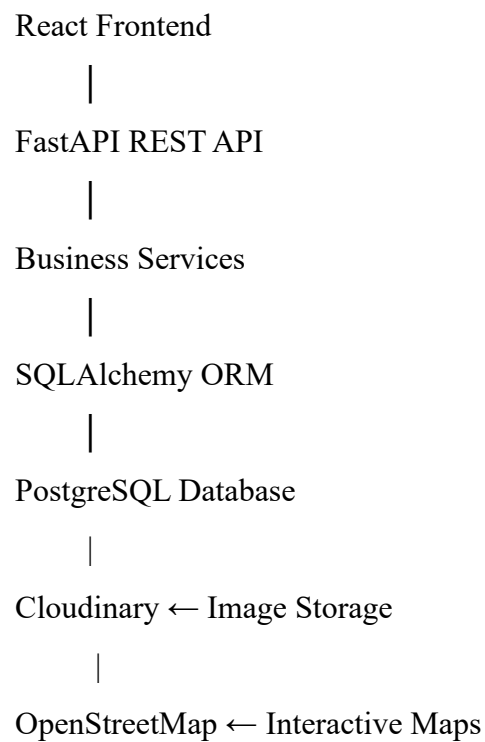
Backend

- FastAPI
- SQLAlchemy
- PostgreSQL
- Alembic
- Pydantic

External Services

- Cloudinary
- OpenStreetMap
- JWT Authentication

6. System Architecture



7. Database Overview

Main Entities

- Users
- Concerns
- Concern Images
- Concern Supports
- Concern History
- Assignments
- Collection Routes
- Collection Points
- Suggestions
- Notifications

8. Assumptions

- Users have internet access.
- Workers use mobile devices to access assigned routes.
- Location services are available for map functionality.
- Municipal authorities manage worker assignments.

9. Constraints

- Web application only.
- Manual worker assignment.
- Requires internet connectivity.
- Image storage handled through Cloudinary.
- OpenStreetMap used for mapping services.

10. Future Enhancements

- Mobile Application
- Smart Bin (IoT) Integration
- AI-based Waste Detection
- Route Optimization Algorithms

- Real-time GPS Tracking
- Push Notifications
- Advanced Analytics Dashboard
- Predictive Waste Collection
- Multi-language Support

11. Expected Outcomes

- Improved waste complaint management.
- Faster issue resolution.
- Better coordination between citizens and municipal authorities.
- Increased transparency in waste collection.
- Efficient monitoring of sanitation workers.
- Digital record of waste management activities.
- Enhanced cleanliness through community participation.